CS F320: Foundations of Data Science

Assignment-1

Due Date: 2355, 27th October
Demo & Viva for 20 mins for each group on mutual convenient time on 28th and 29th October

Provided Files

All files have columns `x,y`. They are fixed: load them directly, and do not generate or modify any data. The true function $\sin(2\pi x)$ is known and may be used for plotting only, never for fitting.

- `poly_train.csv`: $N = 10$ training set (used in P1, P2 and P3).
- `poly_test.csv`: $N = 100$ test set (used in P2 and P3).
- `p4_train_N5.csv`, `p4_train_N10.csv`, `p4_train_N20.csv`, `p4_train_N50.csv`: training sets of size N for P4.

Note. Throughout this assignment, $\text{cond}(A)$ denotes the condition number of matrix A, e.g. via `np.linalg.cond`, which measures numerical stability and optimisation difficulty.

P1: Polynomial Regression — Closed Form vs. the Gradient Descent Family

Objective

Implement polynomial regression both via the closed-form solution and via the full gradient descent family (Batch GD, Stochastic GD, Mini-batch GD) from scratch, verify that all methods agree, and study how the choice of learning rate governs convergence.

Dataset

You are provided with `poly_train.csv`. This file contains $N = 10$ data points (x_i, y_i), where the x_i are evenly spaced in $[0, 1]$ and the target y is generated from a noisy sine wave. Load this dataset directly; do not generate your own data.

Relevant Formulae

- Polynomial model: $y \approx \hat{y} = w_0 + w_1 x + w_2 x^2 + \cdots + w_d x^d = \sum_{j=0}^{d} w_j x^j$. The design matrix $\Phi \in \mathbb{R}^{N \times (d+1)}$ has n-th row $\phi(x_n)^\top = [1, x_n, \ldots, x_n^d]$.
- Objective (SSE): $E(w) = \frac{1}{2} \sum_{n=1}^{N} \big(y(x_n, w) - t_n\big)^2 = \frac{1}{2} \|\Phi w - t\|^2$.
- Gradient: $\nabla E(w) = \Phi^\top (\Phi w - t)$.
- Closed form (normal equations): setting the gradient to zero gives $\Phi^\top \Phi \, w = \Phi^\top t$.

- Batch GD update: $w^{(k+1)} \leftarrow w^{(k)} - \eta \nabla E(w)\big|_{w=w^{(k)}}$, using the full sum over all N points (do not divide by N).
- Stochastic GD: in each epoch, randomly shuffle the data set, then for $n = 1, \ldots, N$ update using only the n-th point's gradient contribution,

$$w^{(k+1)} \leftarrow w^{(k)} - \eta\big(y(x_n, w^{(k)}) - t_n\big)\phi(x_n).$$

- Mini-batch GD: same as SGD, but each update uses the averaged gradient over a mini-batch. Fix the mini-batch size to 3. Since $N = 10$, the last batch of each epoch contains the single remaining sample.
- One epoch is one pass over all N points: one update for Batch GD, $N = 10$ updates for SGD, and 4 updates for Mini-batch GD (batches of sizes 3, 3, 3, 1).
- Fix a random seed for the per-epoch shuffling (e.g. `np.random.default_rng(seed)`) and state the seed you used in your report.

Implementation Note (Numerical Stability for $d = 9$)

For $d = 9$ and $N = 10$, the Vandermonde design matrix Φ is severely ill-conditioned ($\mathrm{cond}(\Phi) \approx 10^7$).

Closed form. Solving the normal equations with an explicit matrix inverse is unstable. Use a robust SVD-based solver (e.g. `np.linalg.lstsq`) to compute the weights directly.

Gradient descent. Running the updates in the original polynomial basis converges astronomically slowly. To make the optimisation tractable without changing the underlying problem, take a thin QR decomposition $\Phi = QR$ (Q has orthonormal columns). Substitute $\alpha = Rw$ and run GD on the well-conditioned problem $E(\alpha) = \frac{1}{2}\|Q\alpha - t\|^2$, starting from $\alpha = 0$. All three variants use the conventions above, now with Q in place of Φ (e.g. Batch GD: $\alpha \leftarrow \alpha - \eta\, Q^\top(Q\alpha - t)$). Recover the original-space weights with $w = R^{-1}\alpha$.

Deliverables

1. Fit polynomials of degree $d = 1, \ldots, 9$ using the closed-form solution and plot the fitted curves against the training points.
2. For $d = 9$, implement Batch GD, Stochastic GD, and Mini-batch GD (mini-batch size 3) from scratch.
3. **Loss curves.** Plot E vs. epoch for all three GD variants on a single set of axes, using one well-chosen η for all three (e.g. $\eta = 0.1$). Use a log-scale y-axis. Run each variant until it has visibly converged (its curve has flattened out), and choose the number of epochs so that all three variants are shown converged in the same plot.
4. **Learning-rate study.** Plot Batch GD (QR-reparametrised, $d = 9$) under three regimes:
   - Too small (e.g. $\eta = 0.01$): slow convergence.
   - Well-chosen ($\eta \in [0.1, 1.0]$): rapid, stable convergence.
   - Too large ($\eta > 2$, e.g. $\eta = 2.1$): divergence. Report the actual weight vector w (and E) after 100 epochs. You may also briefly describe what happens at exactly $\eta = 2$.

   Choose an epoch range that makes the difference between the three regimes clear.

5. **Verification.** Once GD has converged ($d = 9$; run each variant for the same fixed number of epochs long enough that all curves have flattened, e.g. 2000 epochs at $\eta = 0.1$), report $\|w_{\text{GD}} - w_{\text{closed}}\|_2$ for each of the three variants, together with the relative error $\|w_{\text{GD}} - w_{\text{closed}}\|_2 / \|w_{\text{closed}}\|_2$. Note that the entries of w are very large for $d = 9$, so interpret the absolute error accordingly.

6. **Discussion.** Compare the loss curves of Batch, Stochastic and Mini-batch GD. Describe what you observe. In particular, explain (i) why the SGD curve coincides with the Batch GD curve, and (ii) why Mini-batch GD converges more slowly than both. *Hint: look at the rows of Q.* Answer in one short paragraph, referring to your loss-curve plot.

P2: Ridge Regression as a MAP Estimate

Objective

A zero-mean Gaussian prior on the weights, combined with the polynomial-regression likelihood, gives a MAP estimate that is exactly the ridge-penalised least-squares objective below. Use that fact directly. Implement ridge regression in two ways (closed form and gradient descent) and study the effect of the penalty.

Data

Use `poly_train.csv` (the same $N = 10$ training set as P1) and `poly_test.csv` ($N = 100$, same noisy-sine process, x drawn uniformly at random in $[0, 1]$; columns `x,y`). Load the files directly and do not generate your own data. The true function $\sin(2\pi x)$ is known and may be used for plotting only, not for fitting. Use the fixed grid `lams = np.logspace(-10, 2, 49)` for all sweeps. RMSE means $\sqrt{\text{mean squared error}}$ on the given set. λ is defined relative to the unnormalised SSE objective below.

Relevant Formulae

Under a prior $p(w) = \mathcal{N}(0, \alpha^{-1} I)$ and Gaussian likelihood with precision β, the MAP estimate minimises

$$\tilde{E}(w) = \tfrac{1}{2} \sum_{n=1}^{N} \big(t_n - w^\top \phi(x_n)\big)^2 + \frac{\lambda}{2} \|w\|^2, \qquad \lambda = \frac{\alpha}{\beta}.$$

The ridge gradient is $\nabla \tilde{E}(w) = \Phi^\top (\Phi w - t) + \lambda w$, and the closed-form ridge solution is $w = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top t$. The prior and noise precisions α and β enter the solution only through the ratio $\lambda = \alpha/\beta$, so in this problem you work with λ directly and no separate values of α and β are needed.

MAP $\Rightarrow$ Ridge derivation checkpoint. Using Bayes' rule, $p(w \mid t, X) \propto p(t \mid w, X)\, p(w)$. For the Gaussian likelihood, $p(t \mid w, X) \propto \exp\big[-\frac{\beta}{2} \|\Phi w - t\|^2\big]$, and for the zero-mean Gaussian prior, $p(w) \propto \exp\big[-\frac{\alpha}{2} \|w\|^2\big]$. Maximising the posterior is equivalent to minimising

$$\tfrac{1}{2} \|\Phi w - t\|^2 + \frac{\alpha}{2\beta} \|w\|^2,$$

thereby obtaining $\lambda = \alpha/\beta$.

Ridge closed-form derivation checkpoint. Differentiating $\tilde{E}(w) = \frac{1}{2} \|\Phi w - t\|^2 + \frac{\lambda}{2} \|w\|^2$ with respect to w and setting the gradient to zero yields $(\Phi^\top \Phi + \lambda I)\, w = \Phi^\top t$. For implementation, solve this linear system directly; do not form an explicit inverse.

Implementation Note (Gradient Descent)

Do not reuse the QR-reparametrised GD from P1. Run gradient descent on the ridge objective directly in the original polynomial basis, starting from $w = 0$, with updates $w \leftarrow w - \eta\big(\Phi^\top(\Phi w - t) + \lambda w\big)$. The penalty makes the Hessian $\Phi^\top\Phi + \lambda I$ much better conditioned than $\Phi^\top\Phi$, so plain GD becomes practical. Use $\eta = 0.04$.

Deliverables

1. **Closed form.** Solve $(\Phi^\top\Phi + \lambda I)w = \Phi^\top t$ for every λ on the grid, using the $d = 9$ monomial basis $\phi(x) = [1, x, \ldots, x^9]^\top$ (the basis of P1).
2. **GD agreement at a fixed λ.** At $\lambda = 10^{-2}$, run batch GD on the ridge objective in the original basis until $\|w_{\text{GD}} - w_{\text{closed}}\|_2 / \|w_{\text{closed}}\|_2 < 10^{-6}$. Report the number of iterations and plot $\tilde{E}(w)$ vs. iteration. Also report $\text{cond}(\Phi^\top\Phi)$ and $\text{cond}(\Phi^\top\Phi + \lambda I)$, and say in one sentence what the penalty did to the conditioning.
3. **RMSE sweep.** Plot train and test RMSE vs. λ (log x-axis) on the same axes. Mark the noise level $\sqrt{0.01} = 0.1$ as a reference line.

   Selecting $\lambda^\star$. Report $\lambda^\star = \arg\min$ of the test RMSE on the grid, the test RMSE at $\lambda^\star$, and the interval of λ over which test RMSE $\leq 1.05\times$ its minimum. In two sentences, say whether the minimum is sharp or flat, and what happens to the fit on each side of the interval.
4. **Fitted curves.** In one figure with three panels, plot the fit against the training data and $\sin(2\pi x)$ for $\lambda = 10^{-10}$ (overfit), $\lambda = \lambda^\star$, and $\lambda = 10^{-2}$ (underfit). Report $\|w\|_2$ for each of the three fits.

P3: L1-Regularised Regression via Subgradient Descent

Objective

The Lasso objective has no closed-form solution. Implement subgradient descent from scratch, study how the L1 penalty trades off fit against sparsity, and compare it with an L2 (ridge) penalty on the same model.

Data

Use `poly_train.csv` ($N = 10$) and `poly_test.csv` ($N = 100$). Load the files directly and do not generate your own data. The true function $\sin(2\pi x)$ is known and may be used for plotting only, not for fitting. RMSE means $\sqrt{\text{mean squared error}}$ on the given set.

Basis

Let $u = 2x - 1 \in [-1, 1]$. The shifted Legendre polynomials are defined by $P_0(u) = 1$, $P_1(u) = u$ and the recurrence

$$(j+1)\,P_{j+1}(u) = (2j+1)\,u\,P_j(u) - j\,P_{j-1}(u).$$

Use the degree-9 model $\hat{y} = \sum_{j=0}^{9} w_j P_j(2x-1)$, so the design matrix has entries $\Phi_{nj} = P_j(2x_n - 1)$ for $j = 0, \ldots, 9$ (in numpy: `np.polynomial.legendre.legvander(2*x - 1, 9)`).

Why this basis.

- *Same model class.* Each P_j has degree j, so this model spans exactly the same space of degree-9 polynomials as the monomial basis of P1–P2, and the unregularised least-squares fit is the same function.
- *Better conditioning.* The Legendre polynomials are orthogonal on $[-1, 1]$, so the columns of Φ at evenly spaced points are far less correlated than the monomial columns: $\text{cond}(\Phi^\top \Phi) \approx 6 \times 10^2$ here, against $\approx 2 \times 10^{14}$ for monomials.
- *Why it matters for L1.* Unlike ridge, the L1 term adds no curvature that could improve conditioning, so in the monomial basis plain subgradient descent converges extremely slowly for small λ.

Note. The L1 penalty acts on the coefficients of whichever basis is used, so which coefficients become zero, and the Ridge comparison in Deliverable 5, refer to this basis.

Relevant Formulae

- Lasso objective: $E(w) = \frac{1}{2}\|\Phi w - t\|^2 + \lambda \|w\|_1$, with λ defined relative to the unnormalised SSE.
- Subgradient of $|w|$: $\{-1\}$ for $w < 0$, $[-1, 1]$ for $w = 0$, $\{+1\}$ for $w > 0$. (At $w = 0$, every slope in $[-1, 1]$ gives a line through the origin that stays below $|w|$, so each is a valid subgradient.) In the implementation use $\text{sign}(0) = 0$ (`np.sign`).
- Subgradient update: $w \leftarrow w - \eta\big(\Phi^\top(\Phi w - t) + \lambda \text{ sign}(w)\big)$, with sign applied elementwise.
- Ridge, for comparison: $w = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top t$. Solve it as a linear system; do not form an explicit inverse.

Optimality conditions (reference for verification). Write $r = t - \Phi w$ and let Φ_j denote the j-th column of Φ. The Lasso objective is convex, so w is a minimiser if and only if $0 \in \partial E(w)$. The subdifferential of E with respect to w_j is $-\Phi_j^\top r + \lambda\, \partial |w_j|$, which gives, for every j,

$$\Phi_j^\top r = \lambda \text{ sign}(w_j) \;\text{ when } w_j \neq 0, \qquad |\Phi_j^\top r| \leq \lambda \;\text{ when } w_j = 0.$$

In words, a coefficient can be exactly zero only when the correlation of its feature with the residual does not exceed λ. This is the algebraic form of the diamond geometry: the SSE contours first touch the ℓ_1 ball $\|w\|_1 = c$ at a corner on a coordinate axis, producing an exactly zero coefficient, whereas against the circular ℓ_2 ball they touch away from the axes, so L2 shrinks coefficients without zeroing them. You will use these conditions in Deliverable 1 to check your solution.

Implementation Note

- Use $\eta = 0.05$, start from $w = 0$, and run 20,000 iterations for every λ.
- Use the fixed grid `lams = np.logspace(-4, 0, 21)` for all sweeps.
- Subgradient descent is not a descent method: E can rise between steps, so also track the best E seen. This best-so-far E is used only for the convergence plot; report every other quantity (RMSE, coefficient counts, $\|w\|_2$, fitted curves) from the final iterate after 20,000 iterations.
- With a constant step, coefficients chatter around zero with amplitude about $\eta\lambda$ and essentially never equal 0 exactly. Call a coefficient "≈ 0" if $|w_j| < 2\eta\lambda$, and use this rule everywhere.

Deliverables

1. **Convergence and verification at $\lambda = 0.1$.**
   (a) Plot $E(w)$ vs. iteration (log-y) together with the best-so-far E.
   (b) Verify your final w with the optimality conditions: report $\max |\Phi_j^\top r - \lambda \operatorname{sign}(w_j)|$ over the coefficients that are not ≈ 0, and $\max |\Phi_j^\top r|/\lambda$ over the ≈ 0 coefficients, which should be ≤ 1.
   (c) Compare against the reference optimum $E^\star = 0.2625$ and

   $$w^\star \approx [0,\ -0.928,\ 0.045,\ 1.117,\ 0.032,\ -0.201,\ 0,\ -0.026,\ 0,\ 0]$$

   (the exact Lasso solution at $\lambda = 0.1$, computed with a coordinate-descent solver with soft-thresholding and rounded to three decimals), and report $\|w - w^\star\|_2$. Because of the $\approx \eta\lambda$ chatter, do not expect agreement to many digits.
2. **RMSE sweep.** Plot train and test RMSE vs. λ (log x-axis) on the same axes. Mark the noise level $\sqrt{0.01} = 0.1$ as a reference line.
3. **Coefficient path.** Plot each w_j vs. λ (log x-axis), showing the coefficients shrinking toward zero. Also report how many coefficients are ≈ 0 at each λ.
4. **Selecting $\lambda^\star$.** Report $\lambda^\star = \arg\min$ of the test RMSE on the grid, the test RMSE at $\lambda^\star$, and the number and indices of the ≈ 0 coefficients at $\lambda^\star$.
5. **Lasso vs. Ridge.** Fit Ridge in the same Legendre basis on the same grid and find its own best λ by test RMSE. For both methods at their best λ, tabulate test RMSE, $\|w\|_2$ and the number of coefficients with $|w_j| < 0.01$.
6. **Fitted curves.** In one figure, plot the Lasso fit at $\lambda^\star$, the Ridge fit at its own best λ, the training data and $\sin(2\pi x)$.
7. **Discussion.** In one short paragraph, using your coefficient path and the table from Deliverable 5, explain why L1 produces zero coefficients while L2 only shrinks them, and describe what happens to train and test error as λ grows past $\lambda^\star$.

P4: Bayesian Regression — MAP/Ridge Equivalence and Effect of N

Objective

Using the closed-form Bayesian linear regression solution (fixed hyperparameters), this problem asks you to (a) numerically confirm that the MAP point estimate coincides with the mean of the full Bayesian posterior predictive, and (b) study how the posterior predictive uncertainty shrinks as the training-set size N grows.

Data

Load the provided files and do not generate your own data. In each file the x_i are evenly spaced in $[0, 1]$ and the y values are noisy samples of $\sin(2\pi x)$ with noise variance $\sigma^2 = 0.01$.

- `p4_train_N10.csv`: the $N = 10$ training set used for the plots and the equivalence check.
- `p4_train_N5.csv`, `p4_train_N20.csv`, `p4_train_N50.csv`: the additional training sets for the N-sweep.

Relevant Formulae

Polynomial basis (degree 3): $\phi(x) = [1, x, x^2, x^3]^\top$. Φ is the $N \times 4$ design matrix whose n-th row is $\phi(x_n)^\top$.

Posterior over the weights: $p(w \mid t) = \mathcal{N}(w \mid m_N, S_N)$, with

$$S_N^{-1} = \alpha I + \beta\, \Phi^\top \Phi, \qquad m_N = \beta\, S_N \Phi^\top t.$$

Predictive distribution: $p(t \mid x, \mathbf{X}, \mathbf{t}) = \mathcal{N}\big(m(x), s^2(x)\big)$, where

$$m(x) = \phi(x)^\top m_N = \beta\, \phi(x)^\top S_N \sum_{n=1}^{N} \phi(x_n)\, t_n, \qquad s^2(x) = \beta^{-1} + \phi(x)^\top S_N\, \phi(x).$$

MAP/ridge point estimate: $w_{\text{MAP}} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top t$, with $\lambda = \alpha/\beta$. For implementation, solve this as a linear system; do not form an explicit inverse.

Posterior and predictive-uncertainty checkpoint. The Gaussian prior and Gaussian likelihood imply a Gaussian posterior over the weights, with precision $S_N^{-1} = \alpha I + \beta\, \Phi^\top \Phi$. The posterior covariance S_N enters the predictive distribution through $\phi(x)^\top S_N\, \phi(x)$, which is the variance of the prediction $w^\top \phi(x)$ under the posterior. Adding the independent noise of a new observation gives

$$s^2(x) = \underbrace{\beta^{-1}}_{\text{observation noise}} + \underbrace{\phi(x)^\top S_N\, \phi(x)}_{\text{parameter uncertainty}}.$$

As N increases, the data provide more information about the polynomial weights, so the parameter-uncertainty contribution is expected to decrease. The observation-noise term β^{-1} remains as the irreducible noise component of a new observation.

Fixed Hyperparameters

Use noise variance $\sigma^2 = 0.01$ (so $\beta = 1/\sigma^2 = 100$), prior precision $\alpha = 0.05$, and polynomial degree 3. For the N-sweep, keep α and β fixed at these values while N varies.

Deliverables

1. **Predictive plot.** Using `p4_train_N10.csv`, plot the predictive mean $m(x)$ over a dense grid in $[0, 1]$ (e.g. 200 points), shading the 95% predictive interval $m(x) \pm 1.96\, s(x)$, with the noisy training points and the true function $\sin(2\pi x)$ overlaid.
2. **Equivalence check.** Compute w_{MAP} (ridge with $\lambda = \alpha/\beta$) and overlay $\hat{y}(x) = w_{\text{MAP}}^\top \phi(x)$ on the same plot as $m(x)$. Confirm numerically that the two curves coincide by reporting $\|w_{\text{MAP}} - m_N\|_2$.
3. **Effect of N.** Refit the full Bayesian model for each $N \in \{5, 10, 20, 50\}$ using the corresponding file, and plot the predictive interval width $2 \times 1.96\, s(x)$ vs. x for each N on one figure. Show the band tightening as N grows, and include the irreducible-noise floor $2 \times 1.96 \times \beta^{-1/2}$ as a horizontal reference line.
4. **Discussion.** In 2–3 sentences, explain why more data reduce parameter and predictive uncertainty in this way, and distinguish this reduction from the irreducible observation-noise term β^{-1}.